

Can Algebra Replace Transcendentals in Language Models?

Tasmai Keni
tas.ken.rt25@dypatil.edu

Abstract

I ask whether a causal language model can replace several customary transcendental operations in its training path while retaining useful predictive performance. I construct a Transformer with a radical-based activation and normalization, an octic positive attention kernel, rational Cayley position rotations, a strictly proper power score, and an inverse-square-root learning-rate schedule. Its tiled attention and fused vocabulary projection have custom backward passes. A standard Transformer with softmax attention, rotary positions, SwiGLU, and cross-entropy provides the reference. I describe the exact production equations, their real-arithmetic properties, numerical assumptions, and the scope of Lean certificates. In a 20.45-million-parameter WikiText-103 comparison, held-out perplexity was 26.919 versus 25.226 after 100,000 steps. At approximately 125 million parameters, three seeds per architecture processed 2,500,853,760 FineWeb-Edu token positions per run on 16 TPU v4 chips. Mean validation perplexity was 25.774 ± 0.084 for the algebraic model and 25.548 ± 0.043 for the reference (standard errors over seeds). The paired algebraic-minus-reference difference was 0.227 with a 95% Student interval of $[-0.275, 0.728]$. Algebraic throughput was 96.1% of reference throughput. Four zero-shot measurements were close but limited in scope. The comparison demonstrates trainability and near-reference performance for this specific bundled design and budget. It does not isolate individual components or establish equivalence at larger scales.

Keywords: causal language modeling; algebraic attention; proper scoring rules; tiled kernels; formal verification; reproducible systems experiments.

1 Introduction

Modern autoregressive Transformers combine a matrix-product backbone with nonlinear operators that are often treated as fixed ingredients: exponential softmax for attention, trigonometric position rotations, sigmoid-family feed-forward gates, and logarithmic cross-entropy [1, 6, 8]. This paper studies a narrower and empirically testable question than whether those operators are necessary in every model: can one coherent replacement, expressible through arithmetic and inverse square roots in its selected training modules, complete a compute-matched language-model experiment and approach a conventional reference model? Answering that question requires a complete training path. A standalone nonlinearity or an unfused reference implementation cannot establish whether the construction works at the vocabulary size, context length, and hardware budget of a causal language model.

This construction replaces several operators simultaneously. A smooth algebraic linear unit (ALU) and parameter-free algebraic vector normalization (AVN) define the residual blocks. A positive octic kernel supplies causal attention; a nonnegative sink contributes denominator mass but no value vector. Rational Cayley rotations encode positions. A power-score objective, called octic algebraic cross-entropy (OACE) in the code, replaces cross-entropy during training. A rational inverse-square-root schedule completes the selected algebraic optimization path. The production code uses tiled attention and a vocabulary projection fused with OACE, including analytical vector–Jacobian products. The reference model uses learned-scale RMS normalization, rotary positions, SwiGLU, tiled softmax attention, and fused linear cross-entropy. Both use the same tied-embedding width, depth, heads, corpus, token ordering, and run budgets at each comparison scale.

The principal measurement is the completed three-seed, approximately 125-million-parameter experiment. Each model sees the same sequence of sampled FineWeb-Edu token positions for a given seed; each run executes 2,385 full-batch updates, or 2,500,853,760 processed positions. The algebraic-to-reference mean perplexity ratio is 1.0089. Its paired difference interval includes zero. A smaller 20.45-million-parameter WikiText-103 comparison, a preregistered candidate sweep, throughput, numerical diagnostics, and four zero-shot tasks provide context. I report the complete candidate set and per-seed results because a selected winner’s short-run value is not an unbiased estimate of the final-run gap.

Contributions and boundaries. The contribution is an implemented *system* and a measured comparison, rather than a claim that any one primitive is superior. I give exact real-arithmetic identities and show which are covered by Lean 4 certificates; I distinguish those from finite-precision behavior verified numerically. The source audit checks selected Python modules for forbidden calls. It does not inspect every compiler instruction or restrict data preparation or evaluation. The measurements support the tested architecture, dataset, hardware, and budget. They do not prove generic Transformer equivalence, general intelligence, or a universal efficiency advantage.

1.1 Relation to earlier designs

The distinction between *kernel choice* and *tiling strategy* matters. FlashAttention uses exact softmax with IO-aware tiling; FlashAttention-2 improves its work partitioning [2, 3]. The tiled algebraic attention retains quadratic pairwise scores, but its unnormalized numerator and denominator add across key tiles without the online maximum and exponential rescaling required by softmax. This is an arithmetic property of the chosen kernel, not a claim that measured end-to-end execution is faster: the standard reference is faster in the larger experiment. Linear attention methods use feature factorizations to change sequence complexity [4]; the production octic kernel here is evaluated on pairwise scores, so I do not assign it linear complexity. Ring Attention addresses context distributed across devices [5]; the 2,048-token comparison does not evaluate that regime.

Cayley rotations share the relative-position dot-product identity of rotary embeddings [6], although their frequency grid and finite precision recurrence differ. AVN resembles RMS normalization with its learned scales removed [7]. ALU–GLU keeps the multiplicative feed-forward structure investigated for GLU variants [8]. The optimizer retains decoupled weight decay as in AdamW [9]. These connections motivate an honest reference architecture but do not by themselves imply matched function classes or matched training dynamics.

2 Architecture and training objective

Let a batch contain token IDs in $\{0, \dots, V - 1\}$ with causal context length T . At the larger scale, $V = 50,257$, model width $d = 768$, 12 residual blocks, 12 attention heads of width 64, and gated feed-forward width 2,048. Embeddings are tied to the output projection in both arms. Weights are stored in FP32, model activations use bfloat16, and reductions that require it use FP32 accumulation. Both arms use pre-normalized residual blocks with query, key, value, output, and three gated feed-forward matrices. The algebraic arm omits learned normalization scales; the standard arm includes them. Table 1 specifies the paired ingredients.

2.1 Activation, normalization, and residual block

For finite real x define

$$u(x) = x(1 + x^2)^{-1/2}, \quad \text{ALU}(x) = \frac{x}{2}(1 + u(x)). \quad (1)$$

Table 1: The two compared training paths. The entries state implemented choices, not independent ablations.

Component	Algebraic arm	Standard arm
Normalization	AVN, no learned scale	learned-scale RMS normalization
Feed-forward	ALU-GLU	SwiGLU
Position	iterated Cayley 2×2 rotations	trigonometric rotary embeddings
Attention	causal $\rho(s)^8$, sink mass Ω	causal softmax
Training head	fused linear projection plus OACE	fused linear projection plus cross-entropy
Learning-rate schedule	rational inverse-square-root decay	cosine decay
Optimizer	AdamW-type moment update	same AdamW-type moment update

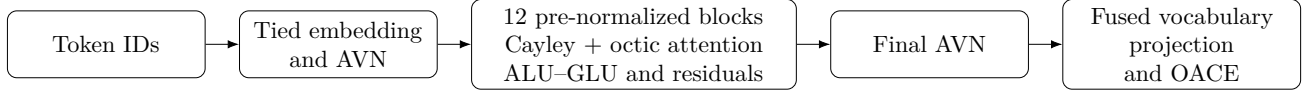


Figure 1: Training path at the larger algebraic scale. Attention and vocabulary projection process tiles; neither materializes a full token-by-vocabulary training matrix.

The implementation caches u for its custom backward pass. For $x < 0$ it evaluates the equivalent expression $\text{ALU}(x) = u(x)(1+x^2)^{-1/2}/[2(1-u(x))]$ to avoid loss of the small negative tail by cancellation in $1+u$. For a feature vector $x \in \mathbb{R}^d$,

$$\text{AVN}_\epsilon(x) = x \left(\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon \right)^{-1/2}, \quad \epsilon = 10^{-5}. \quad (2)$$

Unlike learned-scale RMS normalization, AVN has no trainable gain. The algebraic block applies AVN before query/key/value projection and again before the gated feed-forward sublayer:

$$h_\ell = \text{AVN}_\epsilon(x_\ell), \quad \tilde{x}_\ell = x_\ell + W_o \mathcal{A}(W_q h_\ell, W_k h_\ell, W_v h_\ell), \quad (3)$$

$$g_\ell = \text{AVN}_\epsilon(\tilde{x}_\ell), \quad x_{\ell+1} = \tilde{x}_\ell + W_d[(W_g g_\ell) \odot \text{ALU}(W_u g_\ell)]. \quad (4)$$

Here $\mathcal{A}$ includes the position rotations, attention kernel, and causal mask. The embedding output and final hidden state are also normalized with AVN. The standard block has the same residual and projection layout with learned-scale RMS normalization, rotary positions, SwiGLU, and softmax.

2.2 Cayley positional rotations

For each paired head coordinate and a real frequency parameter w , define

$$R(w) = \frac{1}{1+w^2} \begin{pmatrix} 1-w^2 & -2w \\ 2w & 1-w^2 \end{pmatrix}. \quad (5)$$

The implementation takes a rationally spaced set of frequencies $w_k = (1 + \alpha k)^{-2}$, with $\alpha = (\sqrt{b} - 1)/\max(d_h/2 - 1, 1)$, base $b = 10^4$, pair index k , and head dimension d_h . The square root of the constant base is formed from inverse square root and multiplication. A scan iterates the base cosine/sine-like pair to construct position m and renormalizes each pair after every step. Queries and keys receive the same position-indexed rotation before their dot product. This is a finite-precision recurrence implementing the exact real-arithmetic matrix powers discussed in Section 3.

2.3 Octic causal attention

For head dimension d_h , let $s_{ij} = \langle R_i q_i, R_j k_j \rangle / \sqrt{d_h}$ be the scaled query/key score for key j and query i . The production kernel operates directly on this score. Define the positive radical map and its octic power as

$$\rho(s) = s + \sqrt{1 + s^2}, \quad \kappa(s) = \rho(s)^8. \quad (6)$$

A numerically safer conjugate form $\rho(s) = 1/(\sqrt{1 + s^2} - s)$ is used for negative scores. The production attention output is

$$D_i = \Omega + \sum_{j \leq i} \kappa(s_{ij}), \quad p_{ij} = \begin{cases} \kappa(s_{ij})/D_i, & j \leq i, \\ 0, & j > i, \end{cases} \quad o_i = \sum_{j \leq i} p_{ij} v_j. \quad (7)$$

The selected configuration uses $\Omega = 1$. The sink contributes to the denominator, so $\sum_j p_{ij} \leq 1$; it is equivalent to a zero-valued additional slot with kernel mass Ω . The public primitive `algebraic_softmax` applies AVN to supplied scores before the same kernel. That standalone primitive is used in numerical studies; the production tiled attention path in Eq. (7) does *not* apply that extra row normalization. Algorithm 1 makes the tiled forward and recomputing backward passes explicit.

Algorithm 1 Tiled causal octic attention

Input: rotated queries Q , keys K , values V ; tile widths B_q, B_k ; configured sink mass Ω (1 in the final runs).

Output: attention rows o_i .

1. Partition queries and keys into tiles. For each query row i , set $N_i \leftarrow 0$ and $D_i \leftarrow \Omega$.
 2. For query tile i_{tile} and key tile j_{tile} , skip the tile if $j_{\text{tile}} B_k \geq (i_{\text{tile}} + 1) B_q$. Otherwise compute $s_{ij} = Q_i^\top K_j / \sqrt{d_h}$ and mask padded keys and all $j > i$.
 3. For each visible score, evaluate $\rho(s)$, using its conjugate form for $s < 0$, then form $\kappa(s)$ with three squarings.
 4. Accumulate $N_i \leftarrow N_i + \sum_j \kappa(s_{ij}) v_j$ and $D_i \leftarrow D_i + \sum_j \kappa(s_{ij})$. Use FP32 scratch for reduced-precision value accumulation.
 5. Return $o_i = N_i / D_i$. The sink contributes denominator mass and has zero value.
 6. In the backward pass, recompute tile weights and use Eq. (15) for score cotangents; accumulate value cotangents as $\partial \mathcal{L} / \partial v_j \leftarrow p_{ij} g_i$ and contract score cotangents with Q and K .
-

2.4 Power-score loss and optimizer

The output logits $z = hW^\top$ undergo AVN across vocabulary coordinates with $\epsilon_{\text{vocab}} = 100$, followed by $p_c = \kappa(\text{AVN}_{\epsilon_{\text{vocab}}}(z)_c) / \sum_k \kappa(\text{AVN}_{\epsilon_{\text{vocab}}}(z)_k)$. For a target distribution y and a strictly positive prediction p , the training score is

$$L_\gamma(p, y) = \gamma \left[8 \sum_c y_c p_c^{-1/8} + \frac{8}{7} \sum_c p_c^{7/8} - \frac{64}{7} \sum_c y_c^{7/8} \right], \quad \gamma = 2. \quad (8)$$

Training uses integer target classes; the soft-target formula states the underlying scoring rule. The standard arm trains with cross-entropy. Because the objectives differ, their raw training-loss values and curves are not directly comparable. Both arms are evaluated using the ordinary negative log probability of their own normalized predictive distribution.

Both arms use first and second moment updates with decoupled weight decay, global gradient clipping at norm 1, and bias corrections formed through integer powers. The algebraic schedule is

$$\eta_t = \eta_{\text{max}} \min(1, t/T_w) \left[1 + \left(\frac{\max(0, t - T_w)}{T_d} \right)^2 \right]^{-1/2}, \quad T_d = 0.8 T_{\text{total}}, \quad (9)$$

where T_w is the warmup length. The standard arm uses linear warmup then cosine decay to a selected minimum rate. The base optimizer implementation is shared; the schedules and selected weight decay values differ. The algebraic update order is summarized in Algorithm 2.

Algorithm 2 Algebraic AdamW update

Input: parameters θ_{t-1} , moments m_{t-1}, v_{t-1} , gradient g_t , and step t .

Output: updated parameters and moments.

1. Aggregate replica gradients and clip the global gradient to norm 1, obtaining $\bar{g}_t$.
 2. Update moments: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \bar{g}_t$ and $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \bar{g}_t^2$.
 3. Apply bias corrections $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$; the powers are evaluated by integer multiplication.
 4. Evaluate η_t from Eq. (9).
 5. Form $u_t = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda M \odot \theta_{t-1}$, then set $\theta_t = \theta_{t-1} - \eta_t u_t$. Here M is the configured decay mask.
-

The square-root notation in the update states its mathematical form; the implementation constructs it from reciprocal square root and multiplication, with a zero branch. For the final algebraic runs, $\eta_{\max} = 10^{-3}$, $T_w = 57$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\lambda = 0.01$, and $T_d = 0.8T_{\text{total}}$.

3 Mathematical properties and verification scope

The identities in this section are statements over real numbers and finite vectors under explicit positivity assumptions. They explain why the construction is mathematically coherent, but they do not bound all floating-point rounding errors, optimizer trajectories, or generalization. The repository separates analytic derivations, Lean 4 certificates, numerical tests, and hardware measurements; none substitutes for another.

3.1 Activation and normalization derivatives

With $u = x / \sqrt{1 + x^2}$, differentiation gives

$$\text{ALU}'(x) = \frac{1}{2} + u \left(1 - \frac{1}{2} u^2 \right). \quad (10)$$

This cubic in the cached u needs only additions and multiplications in the custom backward pass. Since $|u| < 1$ for finite x , the derivative is bounded. A loose interval $[-0.05, 1.05]$ suffices for the formally checked polynomial bound; the implementation does not rely on a globally tight constant. In finite precision, the negative branch in Section 2 is consequential: directly forming $1 + u$ for large negative x may round to zero even when the real-valued ALU remains nonzero. Figure 2 plots the analytic derivative.

Write $m = d^{-1} \sum_j x_j^2$ and $\tau = (m + \epsilon)^{-1/2}$. For $\epsilon > 0$, the AVN Jacobian is

$$J_{\text{AVN}}(x) = \tau I - \frac{\tau^3}{d} x x^\top, \quad J_{\text{AVN}}(x)^\top g = \tau \left[g - \frac{x^\top g}{d(m + \epsilon)} x \right]. \quad (11)$$

This is the radial correction implemented in the custom vector–Jacobian product. It also yields $\| \text{AVN}_\epsilon(x) \|_2^2 = d m / (m + \epsilon) \leq d$ and hence $| \text{AVN}_\epsilon(x)_j | \leq \sqrt{d}$. The bound concerns normalized *inputs*; a subsequent learned linear projection can produce unbounded real-valued attention scores without restrictions on its weights. Accordingly, I do not infer a uniform bound on the octic scores from AVN. The ϵ term also breaks exact positive scale invariance: for $\epsilon = 0$ and $\alpha > 0$, $\text{AVN}_0(\alpha x) = \text{AVN}_0(x)$, whereas for fixed positive ϵ the equality is only approximate when $\alpha^2 m \gg \epsilon$.

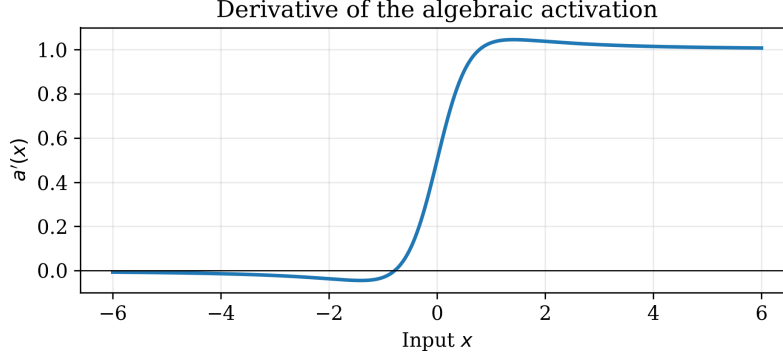


Figure 2: Closed-form ALU derivative from Eq. (10). The curve is a mathematical visualization, not a measured training result.

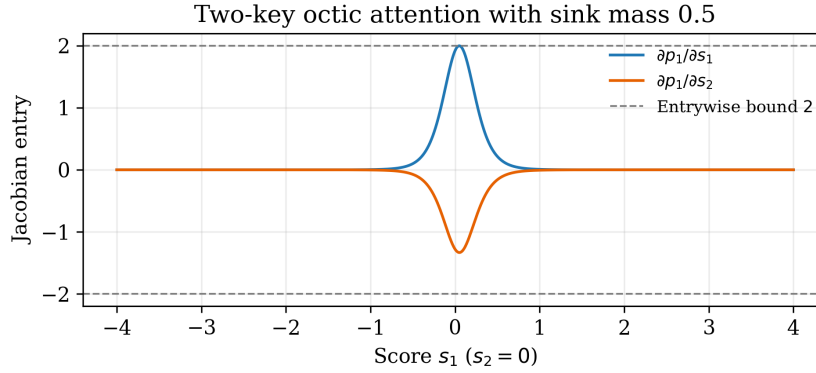


Figure 3: Analytic two-key Jacobian slice, with the indicated sink. Dashed lines denote the entrywise bound derived from Eq. (13).

3.2 Attention positivity, tile addition, and Jacobian

Because $\sqrt{1+s^2} > |s|$, $\rho(s) > 0$ for every finite real s . Moreover

$$\frac{d}{ds} \log \rho(s) = \frac{1}{\sqrt{1+s^2}}, \quad \kappa'(s) = \frac{8\kappa(s)}{\sqrt{1+s^2}}. \quad (12)$$

The logarithm in this displayed proof is an analytic tool; the production algebraic training code does not evaluate it. For the visible keys in one row, differentiating Eq. (7) gives

$$\frac{\partial p_i}{\partial s_j} = \frac{8}{\sqrt{1+s_j^2}} p_i (\mathbf{1}_{i=j} - p_j). \quad (13)$$

Here the row index is suppressed and p_i labels key i . Since $0 \leq p_i \leq 1$ and $\sum_i p_i \leq 1$ when $\Omega \geq 0$, diagonal and off-diagonal entries have magnitude at most 2. For an off-diagonal entry, $p_i p_j \leq 1/4$ follows from $p_i + p_j \leq 1$; for a diagonal entry, $p_i(1 - p_i) \leq 1/4$. Figure 3 shows a two-key slice. This is an *entrywise* bound with respect to the score variables. It is neither a matrix spectral-norm bound nor a bound on derivatives with respect to q or k , which include the score-map Jacobian.

For disjoint visible key tiles $\mathcal{T}_1, \dots, \mathcal{T}_M$, define $N_{it} = \sum_{j \in \mathcal{T}_t} \kappa(s_{ij}) v_j$ and $D_{it} = \sum_{j \in \mathcal{T}_t} \kappa(s_{ij})$. Then

$$o_i = \frac{\sum_{t=1}^M N_{it}}{\Omega + \sum_{t=1}^M D_{it}}. \quad (14)$$

In exact arithmetic, the tile numerators and denominators add associatively. Unlike tiled softmax, no online maximum and rescaling are needed. In floating point, the order of addition can change rounding and large octic powers can overflow; tile associativity is not an equality of all machine execution orders.

The analytical backward path uses $D_i^{(v)} = g_i^\top o_i$ for output cotangent g_i . For visible key j ,

$$\frac{\partial \mathcal{L}}{\partial s_{ij}} = \frac{8p_{ij}}{\sqrt{1 + s_{ij}^2}} \left(g_i^\top v_j - D_i^{(v)} \right). \quad (15)$$

The score cotangent then contracts with keys and queries (including the $1/\sqrt{d_h}$ score scale), and values receive $\sum_i p_{ij} g_i$. The tested implementation handles all three cotangents; a forward-only kernel would not establish trainability.

3.3 Position and scoring-rule identities

Let $a = (1 - w^2)/(1 + w^2)$ and $b = 2w/(1 + w^2)$. Since $(1 - w^2)^2 + 4w^2 = (1 + w^2)^2$, $a^2 + b^2 = 1$. Therefore $R(w)^\top R(w) = I$ and $\det R(w) = 1$. For integer positions m, n ,

$$\langle R(w)^m q, R(w)^n k \rangle = q^\top R(w)^{n-m} k. \quad (16)$$

This establishes relative-position structure in real arithmetic. The implementation’s repeated bfloat16-facing rotation tables are renormalized in FP32 before casting, but finite precision and repeated composition can still drift. No length-extrapolation claim follows from Eq. (16) alone.

The power score in Eq. (8) is a Bregman divergence generated by the strictly convex function $\Phi(p) = -(64/7) \sum_c p_c^{7/8}$ on the positive orthant. Indeed, $\partial \Phi / \partial p_c = -8p_c^{-1/8}$ and $\partial^2 \Phi / \partial p_c^2 = p_c^{-9/8} > 0$, so

$$L_\gamma(p, y) = \gamma \left[\Phi(y) - \Phi(p) - \nabla \Phi(p)^\top (y - p) \right] \geq 0. \quad (17)$$

For interior target distributions, equality holds exactly at $p = y$. For a one-hot target on the boundary, the softmax-style positive parameterization approaches the zero-loss infimum as p approaches that target; it does not attain a zero-probability coordinate at finite logits. The non-local $\sum_c p_c^{7/8}$ term is necessary: dropping it changes the optimum for soft targets. Strict propriety of the score follows from the analytic convexity argument; the present Lean loss module certifies selected Pearson and power-chain algebra, not the full propriety theorem.

3.4 What is machine checked

The Lean library includes algebraic identities for ALU symmetry and its cached derivative polynomial, AVN coordinate bounds, octic powers and attention factors, Cayley orthogonality and determinant, a Pearson divergence expansion, loss power-chain decomposition, optimizer debiasing, and additive tile accumulation. Its proof-coverage document lists the hypotheses of each theorem. Differentiation of the compiled JAX graph, finite-precision reduction behavior, end-to-end optimizer convergence, and benchmark outcomes are outside these certificates. Those are assessed with numerical reference tests and recorded execution, with the narrower interpretation appropriate to each. Appendix C maps the claims to files and records.

4 Implementation and numerical method

The model is implemented in JAX. Static choices such as tile width, sink mass, vocabulary chunk width, and numerical epsilon are configuration values outside traced array arguments. The production algebraic path calls the tiled `algebraic_flash_attention` and `fused_linear_oace` functions. The reference

path calls `standard_flash_attention` and a fused linear cross-entropy. Tests compare kernels with independent dense numerical references, check gradients, and exercise causal masks and nonmultiples of tile dimensions. A local smoke run checks wiring, not hardware performance or paper results.

4.1 Tiled causal attention

The Pallas TPU forward kernel uses query and key tiles and accumulates an unnormalized vector numerator and scalar denominator as in Eq. (14). The query/key matrix product is requested with FP32 preferred output; reduced-precision value matrix products also accumulate into FP32 scratch. For each candidate key tile, the mask compares *absolute token positions*, not just tile numbers. A tile can be skipped only if its first key lies after the last query in the current query tile:

$$j_{\text{tile}}B_k \geq (i_{\text{tile}} + 1)B_q.$$

Inside every included tile, individual keys satisfy $j \leq i$. This distinction matters when query and key tile lengths differ. A second mask zeros weights on padded keys when a valid sequence length is shorter than the compiled tile extent. An interface wrapper pads arbitrary lengths to valid tile multiples and slices the returned output back to the original length. Zeroing padded keys is needed even if their values happen to be zero, because they would otherwise alter the denominator.

The custom VJP uses Eq. (15) and recomputes the weights needed for each cotangent. This trades arithmetic for bounded intermediate storage. The standard fallback instead retains online row maxima and normalizers for tiled softmax, then recomputes weights in its backward pass. A maintained TPU FlashAttention implementation is selected where available, while a tested tiled XLA path supports other platforms. Thus both arms use tiling; the comparison does not handicap the reference with a dense attention matrix. The octic kernel has three explicit squarings after computing ρ : ρ^2 , ρ^4 , and ρ^8 . For $s < 0$, $\rho(s)$ uses a conjugate expression so the small positive result is not formed by subtracting nearly equal numbers.

4.2 Fused vocabulary projection

A naive training head forms an $N \times V$ array of logits for N token positions and vocabulary size $V = 50,257$. The algebraic fused head processes vocabulary chunks (configured width 16,384 for the algebraic model) and accumulates the octic partition sum, target contribution, and reductions needed by its custom VJP. It avoids retaining the complete token-by-vocabulary matrix. Before the vocabulary sweep, the squared logit norm needed by AVN is computed with the Gram identity

$$\|hW\|_2^2 = h(WW^\top)h^\top, \tag{18}$$

which is exact in real arithmetic. The reduced Gram matrix requires $O(d^2)$ storage rather than an $N \times V$ logit array; the matrix products and the streamed vocabulary pass still have substantial compute cost. The implementation factors $p_c^{-1/8}$ into a scalar partition-sum radical and a reciprocal ρ_c , avoiding one radical cascade per class in the fused path. It streams the exact logit-space backward reductions under its arithmetic definition. The standard fused head similarly streams vocabulary chunks using log-sum-exp logic. Evaluation uses a separate chunked path to obtain target log probabilities for either model; logarithms and exponentials used there are explicitly outside the algebraic training-source restriction. Algorithm 3 shows the streamed computation used during training.

Algorithm 3 Fused vocabulary projection and OACE

Input: hidden rows h_i , vocabulary matrix W , target classes y_i , chunk width C , ϵ_{vocab} , and γ .

Output: mean OACE loss and gradients.

1. Compute the vocabulary normalization factor using $\|h_i W\|_2^2 = h_i(WW^\top)h_i^\top$ and $\tau_i = (V^{-1}\|h_i W\|_2^2 + \epsilon_{\text{vocab}})^{-1/2}$.
 2. For each vocabulary chunk c , form normalized logits $x_{ic} = \tau_i h_i W_c$. Accumulate $Z_i = \sum_c \kappa(x_{ic})$ and $B_i = \sum_c \rho(x_{ic})^\top$, while retaining the target class value $\rho(x_{iy_i})$ and the scalar reductions needed by the custom VJP.
 3. Compute $p_{iy_i} = \kappa(x_{iy_i})/Z_i$ and $\sum_c p_{ic}^{7/8} = Z_i^{-7/8} B_i$. Evaluate Eq. (8) for each token and average the losses.
 4. In the backward pass, stream over vocabulary chunks again and accumulate contributions to ∇W_c and ∇h_i from the cached scalar reductions. Do not retain the full token-by-vocabulary logits or probability matrix.
-

4.3 Precision, failure modes, and audit scope

The algebraic primitives promote half-precision inputs to at least FP32 for squared reductions and most internal scalar arithmetic. Output activations and cotangents return to the input dtype. This prevents a common accumulation failure but cannot guarantee the octic kernel stays representable for every finite score. The program assumes finite inputs and finite squared reductions. The experimental safety counters record non-finite updates, loss spikes, and clipped gradient norms during each run; a zero counter means the instrumented event was not observed, not that arbitrary future inputs are safe.

The AST source audit rejects explicit calls to exponential, logarithmic, trigonometric, hyperbolic, sigmoid, and non-integer-power functions in the selected production algebraic modules. Python constants and structural operations such as reciprocal, integer-power multiplication, and `lax.rsqrt` remain permitted. The audit does not establish a theorem about lowering through XLA, instruction selection on a TPU, dataset tokenization, or benchmark evaluation. The standard arm deliberately uses its conventional transcendental operations.

5 Experimental protocol

5.1 Datasets and tokenization

The smaller comparison uses tokenized WikiText-103 [10]. The larger candidate sweep and final runs use a prepared FineWeb-Edu sample [11]. The preparation script reads public Parquet shards, tokenizes text with the GPT-2 BPE vocabulary of 50,257 tokens, and writes memory-mapped arrays. Its configured outputs contain 2.5 billion training tokens, 600 million sweep tokens, and 10 million held-out validation tokens. Training positions are sampled from the relevant token array by a deterministic seed-and-step calculation. A batch example is a contiguous $T + 1$ token span, with the last T tokens shifted as next-token targets. The loader can revisit corpus positions: processed token positions means the sum of positions presented to the optimizer, not the number of unique text tokens. The validation estimate is computed on the same leading, disjoint spans of the held-out array for both architectures. The larger validation call uses ten batches of four sequences at length 2,048, or 81,920 target positions. This is a fixed subset of the 10-million-token validation cache, not the full cache.

For each final run, a model first predicts a normalized distribution for every validation target, then the evaluator averages ordinary $-\log p(y | x)$ over positions and exponentiates. This common metric makes cross-architecture held-out perplexity interpretable despite different training losses. Candidate selection uses the same validation construction. The 20.45-million-parameter pilot uses twenty evaluation batches of 32 sequences at length 512.

5.2 Model sizes, budgets, and hardware

The smaller configuration has width 288, six blocks, six heads, context 512, and gated feed-forward width 768. Counting the tied embedding and the seven projection matrices per block gives 20,445,984 trainable scalars in the algebraic model and 20,450,016 in the standard model; the latter has learned normalization gains. It runs for 100,000 updates with 64 sequences per global batch: $64 \cdot 512 = 32,768$ positions per step and 3,276,800,000 processed positions over the run. One paired comparison was recorded. The larger configuration is described in Section 2; its global batch is 512 sequences of 2,048 positions. All hardware runs use 16 TPU v4 chips across four processes, with a logical mesh of (2, 2, 4) over data, FSDP, and model axis names. The training step has 16 gradient-accumulation microsteps. The larger model has 123,532,032 algebraic and 123,552,000 standard trainable scalars, a difference of 0.016% from normalization gains. Per-seed parameters are initialized from the same seed identifier and both architectures receive the same sampled token starts for that seed. These controls match compute exposure, although they do not make architecture-specific gradients or parameter counts identical.

A full-batch budget is rounded upward. The candidate target of 600,000,000 positions requires $\lceil 600,000,000 / (512 \cdot 2048) \rceil = 573$ updates, or 600,834,048 processed positions. The final target of 2.5 billion requires 2,385 updates, or 2,500,853,760 processed positions. Tables report executed positions rather than substituting requested token counts. Candidate runs together processed 10,815,012,864 positions. Final runs together processed 15,005,122,560 positions.

5.3 Candidate selection and final runs

The candidate matrix was fixed in `configs/` and contains three named choices per architecture. Every choice was run at seeds 42, 43, and 44, for 18 candidate runs. Each candidate combines a learning rate, warmup length, weight decay, moment coefficients, schedule, and (for the algebraic arm) sink mass and score scale. Candidates had to finish their budget, keep validation metrics finite, stay below the specified seed-variation threshold and gradient-norm threshold, record no non-finite updates or loss-spike events, and satisfy the algebraic normalization diagnostic. The eligible candidate with the lowest *mean validation negative log-likelihood* was selected separately for each architecture. The final runs use those frozen configurations. Appendix A gives every candidate’s three perplexities and the selected settings.

The two winners share peak learning rate 10^{-3} , 57 warmup steps, $\beta_1 = 0.9$, $\beta_2 = 0.98$, and norm-1 clipping. The algebraic winner uses weight decay 0.01, sink mass 1, OACE scale 2, and rational decay; the reference uses weight decay 0.1 and cosine decay to 5×10^{-5} . Those different optima are allowed by the equal-candidate selection protocol. They also mean the observed gap belongs to the *bundled optimized arms*, not to a controlled one-component substitution.

5.4 Zero-shot tasks and uncertainty

After the larger runs, I evaluate ARC-Easy test, HellaSwag validation, PIQA validation, and LAMBADA test [12, 13, 14, 15]. Dataset rows are normalized by a checked script and their serialized SHA-256 values are saved with the benchmark records. ARC-Easy, HellaSwag, and PIQA are multiple-choice tasks; each continuation receives its *mean* target-token log probability, and the highest score wins. This choice-length normalization can differ from other evaluations of the same datasets. LAMBADA uses exact match of all token predictions in the final whitespace-delimited target word. Contexts are left-truncated to fit 2,048 tokens, with deterministic padding to compilation buckets. The evaluation script rejects a development-only row-limited dataset when producing the recorded full-task evidence. I report accuracy as correct divided by the recorded task row count.

For a three-seed vector x_s , the reported standard error is $\text{SD}(x_s)/\sqrt{3}$. For the primary paired comparison, I form $d_s = \text{PPL}_{A,s} - \text{PPL}_{S,s}$ and use $\bar{d} \pm t_{0.975,2} \text{SD}(d_s)/\sqrt{3}$, with $t_{0.975,2} = 4.30265$. This

Table 2: Held-out perplexity by experiment. The tuning row reports the selected candidate from each arm; it is selection data. The uncertainty after $\pm$ is one standard error over three seeds.

Experiment	Algebraic	Standard	Ratio	Data and executed budget
20.45M pilot	26.919	25.226	1.0671	WikiText-103; 100,000 updates per arm
125M tuning	47.292 ± 0.215	56.707 ± 1.107	0.8340	FineWeb-Edu; 600,834,048 positions per run
125M final	25.774 ± 0.084	25.548 ± 0.043	1.0089	FineWeb-Edu; 2,500,853,760 positions per run

Table 3: Complete final-run language-model measurements. Throughput is tokens per second, measured on the recorded 16-chip configuration. NLL is the common validation negative log-likelihood.

Architecture	Seed	NLL	Perplexity	Throughput
Algebraic	42	3.25033	25.79898	574,535
Algebraic	43	3.25447	25.90587	574,691
Algebraic	44	3.24327	25.61731	574,312
Standard	42	3.24285	25.60647	597,728
Standard	43	3.23722	25.46279	597,677
Standard	44	3.24155	25.57337	597,994

interval describes seed variation under the fixed data, selection method, hardware, and evaluation subset. With only three seeds it is wide; it does not include uncertainty from changing data source, tokenizer, architecture, or benchmark item sample. No significance claim is inferred from a threshold crossing alone.

6 Results

6.1 Validation language modeling

Table 2 places the single-run pilot, the selected short candidate runs, and the frozen final runs on their separate data and budgets. The smaller pilot reached 26.919 algebraic and 25.226 standard perplexity after 100,000 updates on WikiText-103. This is a useful execution result, but a single seed cannot quantify seed variability. In the 600.8M-position candidate sweep, the selected algebraic configuration reached 47.292 mean perplexity, while the selected standard configuration reached 56.707. The final 2.501B-position runs were much closer: 25.774 versus 25.548. The short-run advantage is not a prediction that the algebraic model retains an advantage after a larger budget; candidate choice, training trajectory, and budget all matter.

The three final algebraic-minus-standard paired differences are 0.193, 0.443, and 0.044 perplexity points for seeds 42, 43, and 44. Their mean is 0.227, standard error 0.117, and 95% Student interval $[-0.275, 0.728]$. The corresponding separate mean intervals are $[25.412, 26.136]$ for the algebraic arm and $[25.361, 25.734]$ for the standard arm. Because both models see the same token order within each seed, the paired differences summarize the comparison more directly than separate intervals. All three point differences favor the reference, but the interval is too broad to establish a nonzero population gap under plausible seed variation. Table 3 exposes the individual results and throughput.

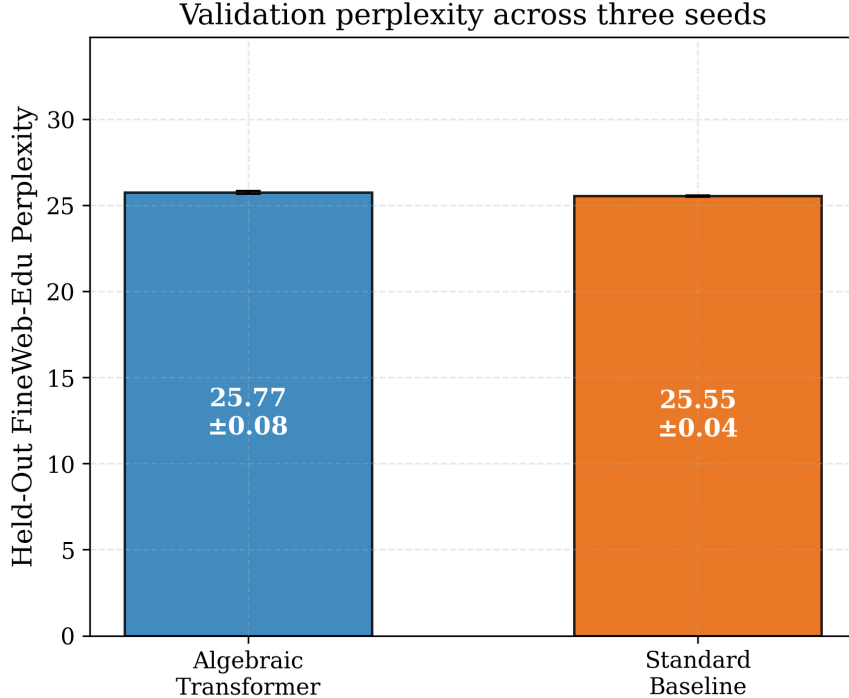


Figure 4: Final-run held-out perplexity, mean over seeds 42–44. Error bars show one standard error; the paired interval in the text is the appropriate interval for the within-seed comparison.

Table 4: Zero-shot accuracy in percent, mean $\pm$ one standard error over three final-run seeds.

Task	Algebraic	Standard	Items per seed
ARC-Easy	42.33 $\pm$ 0.41	42.79 $\pm$ 0.06	2,376
HellaSwag	27.53 $\pm$ 0.08	27.58 $\pm$ 0.02	10,042
PIQA	61.01 $\pm$ 0.28	61.03 $\pm$ 0.62	1,838
LAMBADA	15.87 $\pm$ 0.57	16.44 $\pm$ 0.22	5,153

6.2 Zero-shot task measurements

Table 4 reports seed means and standard errors for all four tasks. All four recorded means favor the standard model, from about 0.02 percentage points on PIQA to 0.57 points on LAMBADA. These differences are small relative to the breadth of claims one might want to make about downstream behavior. Seed-level standard errors do not account for task-item sampling, alternative prompts or scoring rules, or uncertainty caused by possible data overlap. Appendix B gives each seed’s accuracy and the task sizes. The task results therefore support a limited statement about this evaluation protocol, not a general capability equivalence claim.

6.3 Throughput and numerical diagnostics

Mean final-run throughput was approximately 574,513 tokens/s for the algebraic model and 597,799 tokens/s for the standard model, a ratio of 0.961. The corresponding mean elapsed times were approximately 4,353 seconds and 4,183 seconds per run. These are observed values on the stated 16-chip TPU v4 slice. The smaller pilot recorded 2.340M and 2.477M tokens/s, respectively, a ratio of 0.945.

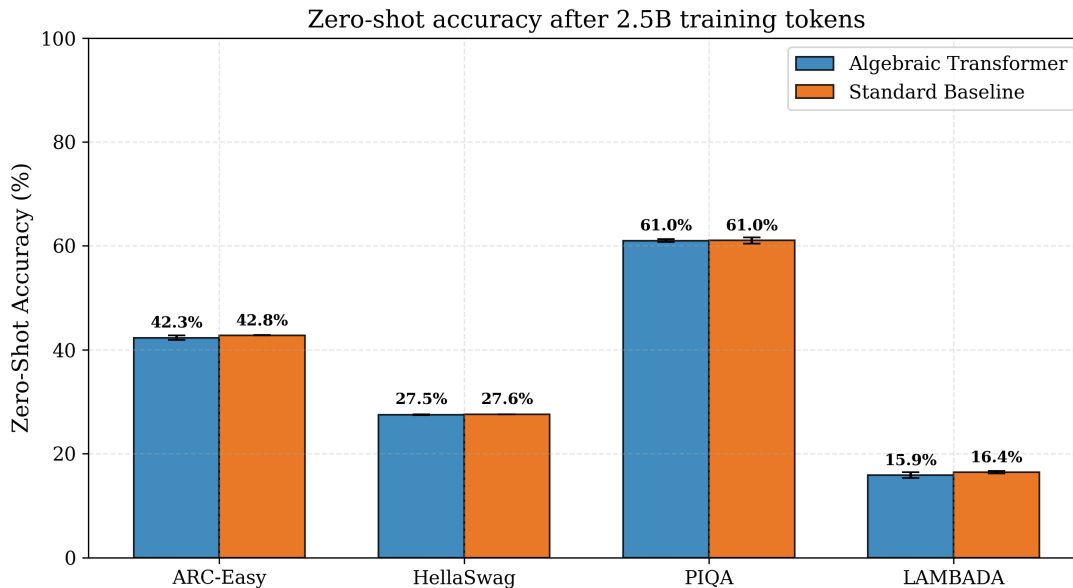


Figure 5: The four recorded zero-shot accuracies after the final runs. Bars show seed means and one standard error. Dataset splits and scoring rules are given in Section 5.

Throughput depends on configuration, compilation, and hardware; no universal kernel-speed conclusion follows. Figure 6 also plots the step time implied by measured tokens per second.

The six final runs recorded zero non-finite updates and zero loss-spike events under the run’s counters; their reported peak post-clipping gradient norm was approximately 1. The algebraic normalization probes at twelve block inputs and the final output had recorded second moments close to one. Those probes are post-normalization diagnostics, so they primarily verify the implemented AVN effect on the sampled validation input. They do not prove that unnormalized residual states have unit variance or that arbitrarily deep stacks remain stable. Figure 7 plots the probes.

6.4 Evidence status

The packaged run JSONs are immutable historical measurements from a recorded source snapshot, with environment and source SHA-256 fields. The current repository has since been reorganized and this manuscript has been expanded. The original numerical leaves can be verified against their Git blobs through the origin manifest, but a historical PASS label is not a fresh hardware PASS for the present source tree. Regenerating current-source hardware evidence would require rerunning the complete candidate and final protocol. No new TPU measurement was made for this manuscript revision.

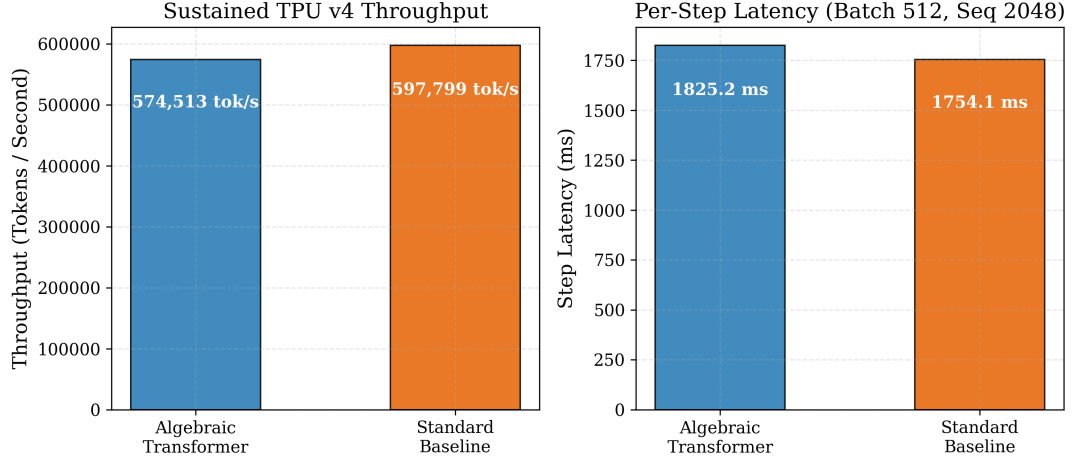


Figure 6: Recorded final-run throughput and the corresponding batch-time calculation on the 16-chip TPU v4 setup.

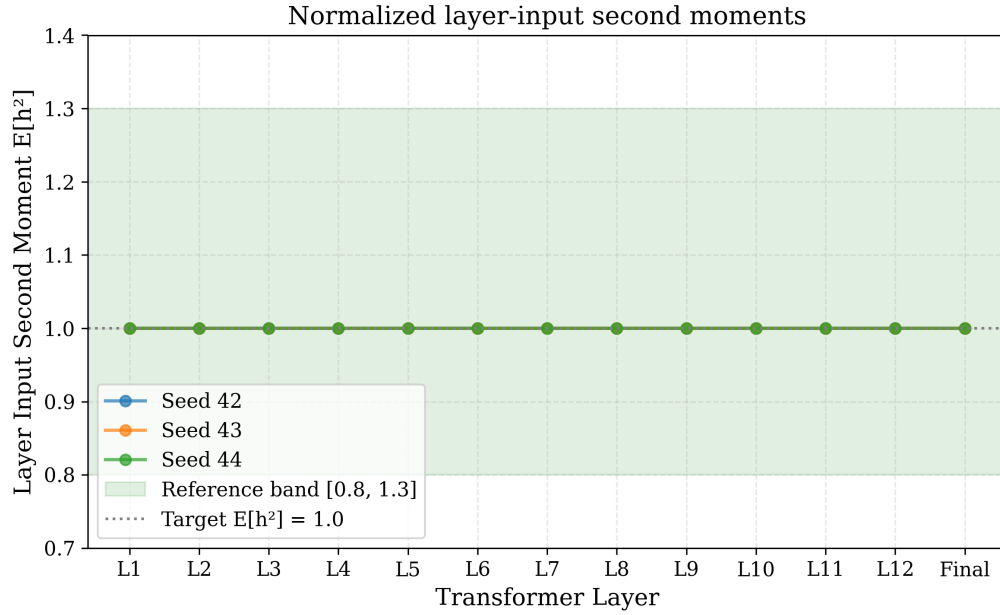


Figure 7: Recorded second moments after normalization at each block input and at the final hidden state in the three algebraic runs. The shaded band is the run’s inspection range, not a confidence interval.

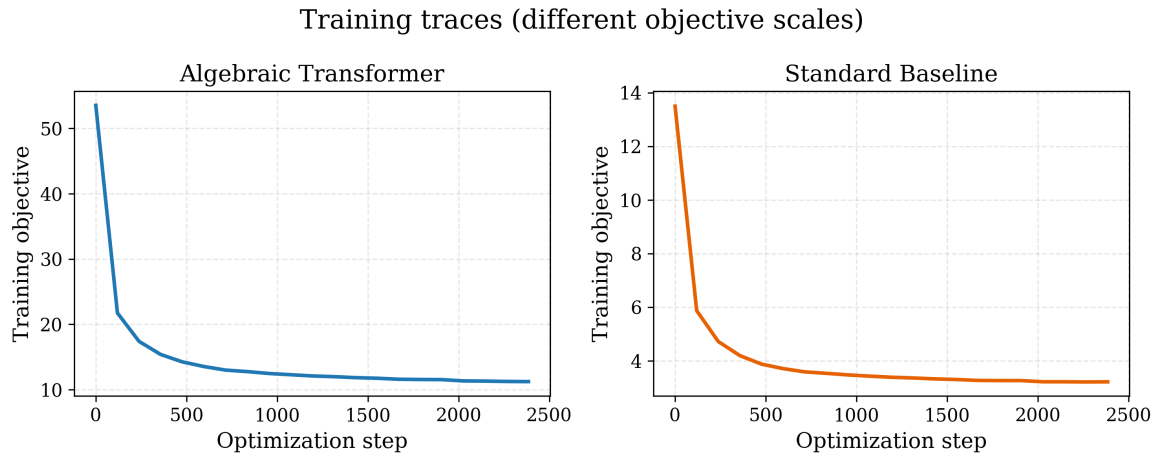


Figure 8: Recorded training traces for the different training objectives. Axes are separate because OACE and cross-entropy have different definitions and numerical scales. Faint curves are seeds; solid curves are their means.

7 Discussion

The final experiment demonstrates that a complete algebraic training path can run at this scale and reach held-out perplexity close to a conventional reference under matched token exposure. It also shows a concrete cost: on the measured TPU slice, the algebraic arm was about 3.9% slower end to end. Replacing exponentials and logarithms in selected source modules is therefore not, by itself, evidence of higher throughput. The fused head and attention kernels make the experiment feasible; they do not guarantee speed superiority over maintained softmax kernels.

The comparison is intentionally bundled. The normalization, activation, position representation, attention kernel, training score, and schedule change together. The candidate sweep selects different weight decays and decay schedules. A factorial or targeted ablation would be required to identify which substitution contributes to the residual perplexity gap or throughput difference. The candidate sweep also reused the validation subset for selection and reporting, so its winning values can be optimistic. The final runs constitute a separate, longer training budget under frozen settings, but still use the same held-out validation construction.

The mathematical identities illuminate mechanism and numerical design. Positive octic weights permit additive tile accumulation; Cayley matrices give exact real-arithmetic relative rotations; the power score is strictly proper for interior targets. These properties do not establish optimization convergence, calibration, long-context behavior, or cross-dataset transfer. The Lean certificates cover stated algebraic subclaims and cannot turn empirical outcomes into theorems. Similarly, a source AST audit is a syntactic restriction on identified modules, not a hardware trace of every instruction.

Interpretation is limited by three seeds, an 81,920-position validation subset, a one-pair pilot, and four zero-shot tasks evaluated under one prompt protocol. The runs use a fixed FineWeb-Edu sample and a 2.5B-position budget; benchmark contamination was not audited. The repository includes source and records, but not prepared token arrays or training checkpoints.

7.1 Future work

The first priority is to separate the bundled design choices. Targeted ablations of ALU, AVN, Cayley positions, octic attention, OACE, and the rational decay schedule should hold token order, evaluation, and tuning effort comparable wherever possible.

Broader measurements are needed as well: more seeds, evaluation on the full validation cache, independent corpus samples, and a contamination audit. Longer contexts and larger models will show whether quality and numerical stability persist beyond the present setting.

Kernel profiles on current accelerators should report compilation, communication, memory traffic, and end-to-end time. On the theory side, extending the scoring-rule and tiled-backward proofs, with finite-precision bounds under explicit assumptions, would clarify the limits of the real-arithmetic analysis.

8 Conclusion

A 12-block causal Transformer using arithmetic and inverse-square-root primitives in its selected training source completed three large runs and a smaller pilot. Its mean final held-out perplexity was 25.774, compared with 25.548 for the compute-matched reference, and its measured throughput was 96.1% of the reference. The paired three-seed interval is compatible with a range of small differences. The result supports feasibility for this implemented design under the tested conditions. Determining which substitutions drive the remaining quality and throughput differences will require the controlled comparisons and broader measurements described above.

A Complete candidate sweep

The matrix in Table 5 contains every prespecified candidate and its three recorded validation perplexities. Each cell is the endpoint of a 573-update run, or 600,834,048 processed positions. The selected candidate in each arm had the lowest eligible mean validation negative log-likelihood. Perplexity is reported here for interpretability; selecting by mean perplexity is not algebraically identical to selecting by mean NLL, although both select the same candidate in these records. The 18 runs are not 18 independent architecture comparisons: they include hyperparameter alternatives and share validation data.

Table 5: All candidate perplexities; the selected entries are in bold.

Arm	Candidate	Seed 42	Seed 43	Seed 44	Mean
Algebraic	reference	62.495	62.936	62.108	62.513
Algebraic	lower LR	107.870	105.904	108.216	107.330
Algebraic	high LR	47.243	47.686	46.947	47.292
Standard	reference	79.096	78.171	79.688	78.985
Standard	lower LR	127.290	126.242	126.450	126.661
Standard	high LR	58.658	56.638	54.826	56.707

The original candidate configuration file records all schedule and regularization fields. For compactness, Table 6 gives the parameters that vary across candidates. Every algebraic candidate uses rational decay and every standard candidate uses cosine decay. All use norm-1 clipping, $\beta_1 = 0.9$, and the same three seeds. The unused standard-arm sink and score-scale fields in the serialized configuration do not change standard softmax or cross-entropy.

Table 6: Candidate settings. LR is peak learning rate; WD is decoupled weight decay; Ω and γ apply only to the algebraic arm.

Arm	Candidate	LR	Warmup	WD	β_2	Ω	γ
Algebraic	reference	$6 \cdot 10^{-4}$	28	.01	.99	.5	2.0
Algebraic	lower LR	$3 \cdot 10^{-4}$	28	.05	.99	.25	1.5
Algebraic	high LR	10^{-3}	57	.01	.98	1.0	2.0
Standard	reference	$6 \cdot 10^{-4}$	28	.01	.99	—	—
Standard	lower LR	$3 \cdot 10^{-4}$	28	.05	.99	—	—
Standard	high LR	10^{-3}	57	.10	.98	—	—

B Per-seed benchmark measurements

Table 7 contains the six benchmark records. Scores are percentages. The row counts are 2,376 for ARC-Easy, 10,042 for HellaSwag, 1,838 for PIQA, and 5,153 for LAMBADA. Their source split, serialized row count, and SHA-256 are stored with each run record. Displaying seed-level values prevents a mean from obscuring a larger seed-specific departure, notably on LAMBADA and PIQA.

C Derivations and proof coverage

The sink as missing mass. For any causal row, positivity gives $D_i > 0$ if $\Omega > 0$. The visible-key weights sum to $\sum_{j \leq i} p_{ij} = 1 - \Omega/D_i$. This is the quantity referred to as omitted attention mass. For $\Omega = 0$, all mass is assigned to visible keys in exact arithmetic. For $\Omega > 0$, the output is a weighted

Table 7: Zero-shot accuracy by architecture and seed, in percent.

Arm	Seed	ARC-Easy	HellaSwag	PIQA	LAMBADA
Algebraic	42	41.625	27.365	60.446	16.515
Algebraic	43	43.056	27.614	61.317	16.379
Algebraic	44	42.298	27.604	61.262	14.729
Standard	42	42.887	27.604	60.229	16.845
Standard	43	42.677	27.594	60.609	16.398
Standard	44	42.803	27.554	62.242	16.088

average with an implicit zero vector of weight Ω/D_i . If every visible value satisfies $\|v_j\|_2 \leq M$, then

$$\|o_i\|_2 \leq \sum_{j \leq i} p_{ij} \|v_j\|_2 \leq M \left(1 - \frac{\Omega}{D_i}\right) \leq M.$$

This bounds one attention output under finite real scores; it does not bound residual states after learned projections.

Exact tile decomposition. Let $\mathcal{T}_1, \dots, \mathcal{T}_M$ be a disjoint partition of the visible keys, and write $N_{it} = \sum_{j \in \mathcal{T}_t} \kappa(s_{ij}) v_j$ and $D_{it} = \sum_{j \in \mathcal{T}_t} \kappa(s_{ij})$. Since the tiles partition the visible indices,

$$\sum_t N_{it} = \sum_{j \leq i} \kappa(s_{ij}) v_j, \quad \sum_t D_{it} = \sum_{j \leq i} \kappa(s_{ij}).$$

Substitution gives $o_i = (\sum_t N_{it})/(\Omega + \sum_t D_{it})$, exactly the dense real-arithmetic result. Floating-point reductions can round differently when their order changes, and finite octic powers can overflow.

Attention Jacobian and vector–Jacobian product. For one row let $D = \Omega + \sum_j \kappa(s_j)$ and $p_i = \kappa(s_i)/D$. The quotient rule gives

$$\frac{\partial p_i}{\partial s_j} = \frac{\kappa'(s_j)}{\kappa(s_j)} p_i (\mathbf{1}_{i=j} - p_j).$$

Using $\kappa'(s)/\kappa(s) = 8/\sqrt{1+s^2}$ from Eq. (12) yields Eq. (13). For $o_i = \sum_j p_{ij} v_j$ and output cotangent g_i , the chain rule gives Eq. (15). The formula includes the dependence of the normalizer on every visible score; it is taken with respect to scores before contracting with the scaled query–key products.

Cayley orthogonality and relative positions. With $a = (1 - w^2)/(1 + w^2)$ and $b = 2w/(1 + w^2)$, $(1 - w^2)^2 + 4w^2 = (1 + w^2)^2$ implies $a^2 + b^2 = 1$. Thus $R = \begin{pmatrix} a & -b \\ b & a \end{pmatrix}$ satisfies $R^\top R = I$ and $\det R = 1$. For integer positions m, n ,

$$(R^m q)^\top (R^n k) = q^\top (R^m)^\top R^n k = q^\top R^{-m} R^n k = q^\top R^{n-m} k.$$

This is an exact real-arithmetic identity. The finite-precision recurrence can drift, so the identity alone gives no arbitrary-length stability or extrapolation guarantee.

OACE gradient and properness. Differentiating Eq. (8) with respect to an interior prediction coordinate yields

$$\frac{\partial L_\gamma}{\partial p_c} = \gamma \left(p_c^{-1/8} - y_c p_c^{-9/8} \right).$$

At $p = y$ every coordinate derivative is zero. The diagonal Hessian of Φ is positive, so the Bregman inequality in Eq. (17) is strict for unequal interior arguments. The score is thus proper for interior soft targets. This derivation uses real calculus, whereas the Lean module proves selected algebraic identities and a reciprocal-eighth radical chain.

Source-to-claim map. The mapping in Table 8 states what a reader can verify from code or certificates. An entry under check is not evidence for claims beyond its stated scope. The complete theorem names and assumptions appear in `formal/PROOF_COVERAGE.md`.

Table 8: Verification layers for representative claims.

Claim	Source or certificate	Check and limit
ALU formula and cached VJP	<code>src/primitives.py</code> , <code>Gate.lean</code>	Real polynomial identities and numerical VJP references; not floating-point proof
AVN coordinate bound	<code>src/primitives.py</code> , <code>Composition.lean</code>	Real bound under explicit ϵ assumptions; not arbitrary-depth stability
Octic attention factors and tile sum	<code>pallas_afa.py</code> , <code>Kernel.lean</code>	Real factor identities plus causal/gradient tests; not universal overflow freedom
Cayley orthogonality	<code>src/attention.py</code> , <code>Cayley.lean</code>	Exact real identity; finite-precision recurrence tested separately
Loss power chain and Pearson identities	<code>src/loss.py</code> , <code>Loss.lean</code>	Algebraic subclaims; full OACE propriety is the analytic argument above
Source operation restriction	<code>scripts/audit_primitives.py</code>	Selected Python AST calls only; compiler and evaluation excluded
Hardware outcomes	<code>results/pretraining/</code> and origin manifest	Historical measured values and source hashes; not a current-source PASS

The Lean file names in the table are under `formal/AlgebraicTheory/`; the Pallas attention file is under `src/kernels/`.

D Reproducibility and artifact status

The project repository at <https://github.com/tasmaikeni13/algebraic-transformer> contains the model and kernel source, candidate matrix, numerical tests, Lean development, run records, and figure-generation scripts. The manuscript source is maintained separately. Dependencies are listed in `requirements.txt`, with TPU additions in `requirements-tpu.txt`. To check record origins, run `pythonscripts/verify_recorded_results.py` from the repository root. Each public JSON result links to its original Git blob, and each run includes a source-hash map.

Prepared token arrays and training checkpoints are not included. A new TPU study should start from a clean source snapshot and regenerate the candidate sweep, final runs, benchmarks, and hardware records. Existing measurements remain historical results rather than evidence from a new run.

Acknowledgments

I thank Google’s TPU Research Cloud (TRC) for providing access to the v4-32 TPU slice (16 TPU v4 chips) used in this study.

References

- [1] Ashish Vaswani et al. Attention Is All You Need. 2017. <https://arxiv.org/abs/1706.03762>.
- [2] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. <https://arxiv.org/abs/2205.14135>.
- [3] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. <https://arxiv.org/abs/2307.08691>.
- [4] Angelos Katharopoulos et al. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. 2020. <https://arxiv.org/abs/2006.16236>.
- [5] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring Attention with Blockwise Transformers for Near-Infinite Context. 2023. <https://arxiv.org/abs/2310.01889>.
- [6] Jianlin Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. <https://arxiv.org/abs/2104.09864>.
- [7] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. 2019. <https://arxiv.org/abs/1910.07467>.
- [8] Noam Shazeer. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>.
- [9] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. 2017. <https://arxiv.org/abs/1711.05101>.
- [10] Stephen Merity et al. Pointer Sentinel Mixture Models. 2016. <https://arxiv.org/abs/1609.07843>.
- [11] Guilherme Penedo et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. 2024. <https://arxiv.org/abs/2406.17557>.
- [12] Peter Clark et al. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. 2018. <https://arxiv.org/abs/1803.05457>.
- [13] Rowan Zellers et al. HellaSwag: Can a Machine Really Finish Your Sentence? 2019. <https://arxiv.org/abs/1905.07830>.
- [14] Yonatan Bisk et al. PIQA: Reasoning about Physical Commonsense in Natural Language. 2019. <https://arxiv.org/abs/1911.11641>.
- [15] Denis Paperno et al. The LAMBADA Dataset: Word Prediction Requiring a Broad Discourse Context. 2016. <https://arxiv.org/abs/1606.06031>.